Team: Analytics Shield

DNS Anomaly Detection Using Machine Learning

```
# ── Standard library & third-party imports ──────────────────────────
import os
import time
import math
import warnings
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
import seaborn as sns

from collections import Counter

# scikit-learn
from sklearn.preprocessing import StandardScaler, LabelEncoder
from sklearn.feature_selection import VarianceThreshold
from sklearn.decomposition import PCA
from sklearn.model_selection import train_test_split
from sklearn.tree import DecisionTreeClassifier
from sklearn.ensemble import RandomForestClassifier
from sklearn.svm import SVC
from sklearn.neural_network import MLPClassifier
from sklearn.metrics import (accuracy_score, precision_score,
                             recall_score, f1_score, confusion_matrix)

warnings.filterwarnings('ignore')

# Reproducibility seed
SEED = 42
np.random.seed(SEED)

# Ensure results directory exists
os.makedirs('results', exist_ok=True)

print('All imports successful.')
```

Output: All imports successful.

Section 1 - Data Loading & Combining

```
# ── Load DGA (malicious) domains ────────────────────────────────────
# Skip lines that start with '#' (file header / comments)
dga_df = pd.read_csv(
```

```
    'dga-domain.txt',
    sep='\t',
    comment='#',
    header=None,
    names=['dga_family', 'domain', 'start_date', 'end_date'],
    engine='python',
    nrows=20000
)

# Keep only the domain column and assign malicious label
dga_domains = dga_df[['domain']].copy()
dga_domains['label'] = 1

print(f'DGA domains loaded   : {len(dga_domains):,}')
print(dga_domains.head(3))
```

Output:

```
 DGA domains loaded   : 20,000

          domain  label
0   jkybwgxpfr.com      1
1       scxjfe.com      1
2  xmkrwnrrtes.com      1
```

```
# ── Load top-1M (benign) domains — first 50,000 rows only ──────────────────
legit_df = pd.read_csv(
    'top-1m.csv',
    header=None,
    names=['rank', 'domain'],
    nrows=50000
)

# Keep only the domain column and assign benign label
legit_domains = legit_df[['domain']].copy()
legit_domains['label'] = 0

print(f'Legitimate domains loaded: {len(legit_domains):,}')
print(legit_domains.head(3))
```

Output:

```
Legitimate domains loaded: 50,000
```

Team: Analytics Shield

```
       domain  label
0   google.com     0
1  youtube.com     0
2  facebook.com    0
```

```
# ── Combine, shuffle, and reset index ──────────────────────────────────
df = pd.concat([dga_domains, legit_domains], ignore_index=True)
df = df.sample(frac=1, random_state=SEED).reset_index(drop=True)

print(f'\nCombined dataset shape : {df.shape}')
print('\nClass distribution:')
print(df['label'].value_counts().rename({0: 'Benign (0)', 1: 'Malicious (1)'}))
```

Output:

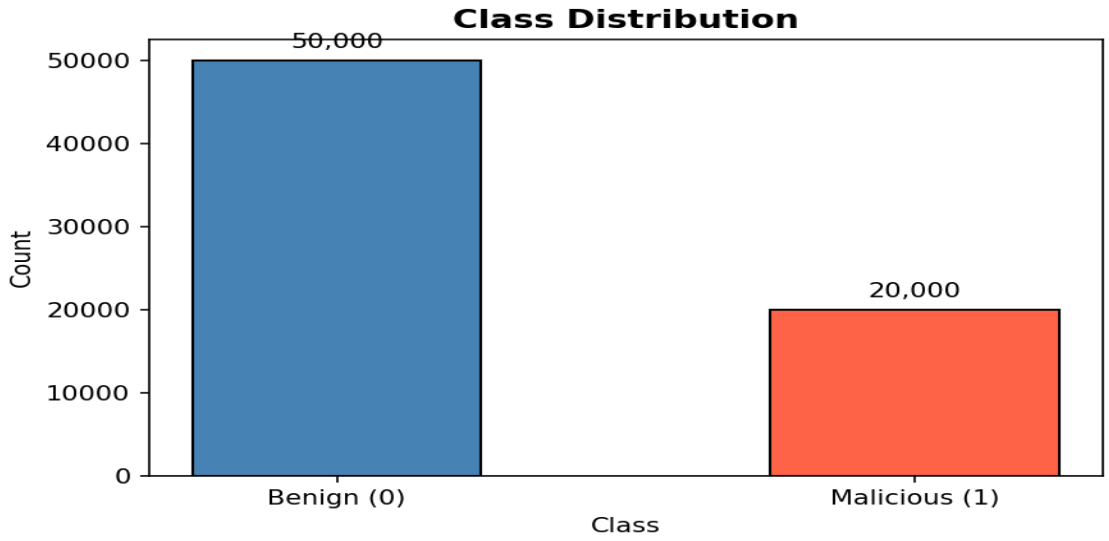
```
Combined dataset shape : (70000, 2)

Class distribution:
label
Benign (0)      50000
Malicious (1)   20000
Name: count, dtype: int64
```

Section 2 - Exploratory Data Analysis (EDA)

```
# ── Helper: Shannon entropy of a string ────────────────────────────────
def shannon_entropy(s: str) -> float:
    """Compute Shannon entropy (bits) of a string."""
    if not s:
        return 0.0
    counts = Counter(s)
    n = len(s)
    return -sum((c / n) * math.log2(c / n) for c in counts.values())

# Compute length and entropy on the full domain string for EDA only
df['eda_length']  = df['domain'].astype(str).apply(len)
df['eda_entropy'] = df['domain'].astype(str).apply(shannon_entropy)

print('EDA columns added.')
```

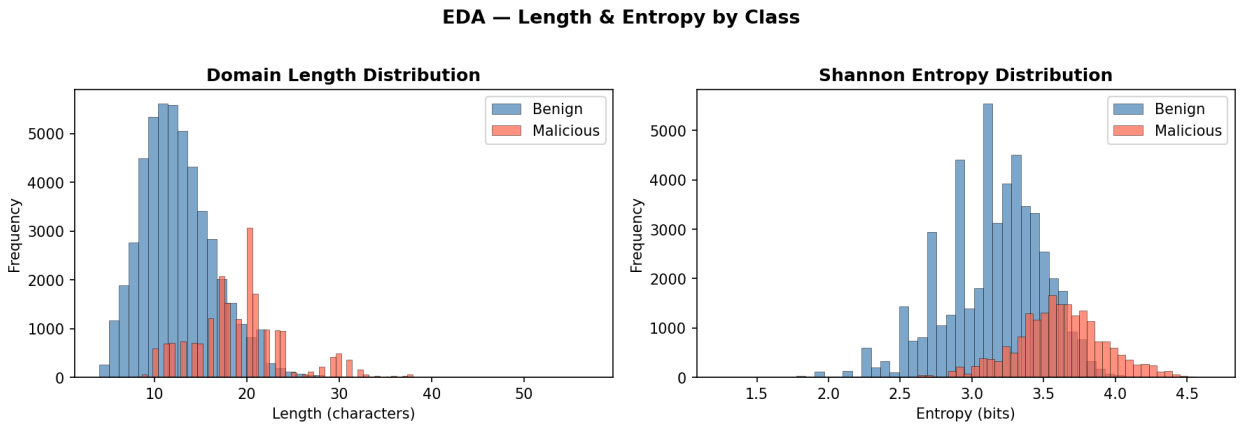
Output: EDA columns added.

```
# ── Figure 1: Class distribution bar chart ──────────────────────────────
fig, ax = plt.subplots(figsize=(6, 4))
counts = df['label'].value_counts().sort_index()
bars = ax.bar(['Benign (0)', 'Malicious (1)'], counts.values,
              color=['steelblue', 'tomato'], edgecolor='black', width=0.5)
for bar, val in zip(bars, counts.values):
    ax.text(bar.get_x() + bar.get_width() / 2, bar.get_height() + 1000,
            f'{val:,}', ha='center', va='bottom', fontsize=10)
ax.set_title('Class Distribution', fontsize=13, fontweight='bold')
ax.set_ylabel('Count')
ax.set_xlabel('Class')
plt.tight_layout()
plt.savefig('results/01_class_distribution.png', dpi=150)
plt.show()
print('Saved: results/01_class_distribution.png')
```




```
# ── Figure 2: Domain length histograms per class ────────────────────────
fig, axes = plt.subplots(1, 2, figsize=(12, 4))
label_map = {0: ('Benign', 'steelblue'), 1: ('Malicious', 'tomato')}

for label, (name, color) in label_map.items():
    subset = df[df['label'] == label]['eda_length']
    axes[0].hist(subset, bins=50, alpha=0.7, color=color, label=name, edgecolor='black',
linewidth=0.3)

axes[0].set_title('Domain Length Distribution', fontweight='bold')
axes[0].set_xlabel('Length (characters)')
```

```
axes[0].set_ylabel('Frequency')
axes[0].legend()

# ── Figure 2b: Shannon entropy histograms per class ──────────────────
for label, (name, color) in label_map.items():
    subset = df[df['label'] == label]['eda_entropy']
    axes[1].hist(subset, bins=50, alpha=0.7, color=color, label=name, edgecolor='black',
linewidth=0.3)

axes[1].set_title('Shannon Entropy Distribution', fontweight='bold')
axes[1].set_xlabel('Entropy (bits)')
axes[1].set_ylabel('Frequency')
axes[1].legend()

plt.suptitle('EDA - Length & Entropy by Class', fontsize=13, fontweight='bold', y=1.02)
plt.tight_layout()
plt.savefig('results/02_eda_length_entropy.png', dpi=150, bbox_inches='tight')
plt.show()
print('Saved: results/02_eda_length_entropy.png')
```




```
# ── Print 5 sample domains from each class ──────────────────────────
print('=== 5 Sample BENIGN domains ===')
print(df[df['label'] == 0]['domain'].sample(5, random_state=SEED).tolist())

print('\n=== 5 Sample MALICIOUS domains ===')
print(df[df['label'] == 1]['domain'].sample(5, random_state=SEED).tolist())
```

Output:

```
 === 5 Sample BENIGN domains ===

['mlstatic.com', 'easynutritionplans.com', 'copaair.com', 'securitylab.ru', 'obi.at']
```

Team: Analytics Shield

=== 5 Sample MALICIOUS domains ===
['midydcmulixgxf.com', 'gittunsdnrpckputu.tw', 'amsovuenr.ddns.net',
'qntvrdqecknjhxkkr.com', 'hmirveqwnuqt.jp']

Section 3 - Feature Extraction

```
# ── TLD stripping helper ──────────────────────────────────────────────
def strip_tld(domain: str) -> str:
    """Return the second-level domain (SLD) by removing the last dot-segment.

    Examples:
      google.com  -> google
      abc.co.uk   -> abc.co   (we only remove the final segment)
    """
    parts = str(domain).strip().lower().split('.')
    # If there is more than one part, remove the last (TLD); otherwise keep as-is
    return '.'.join(parts[:-1]) if len(parts) > 1 else parts[0]


# ── Feature extraction functions ──────────────────────────────────────
VOWELS = set('aeiou')
CONSONANTS = set('bcdfghjklmnpqrstvwxyz')

def extract_features(sld: str) -> dict:
    """Extract lexical features from a second-level domain string."""
    s = str(sld).lower()
    n = len(s) if len(s) > 0 else 1   # avoid division by zero

    # Length
    length = len(s)

    # Shannon entropy
    entropy = shannon_entropy(s)

    # Character-type ratios
    digits     = sum(c.isdigit() for c in s)
    vowels     = sum(c in VOWELS for c in s)
    consonants = sum(c in CONSONANTS for c in s)

    digit_ratio     = digits / n
    vowel_ratio     = vowels / n
    consonant_ratio = consonants / n
```

```
    # Unique character count
    unique_chars = len(set(s))

    # Maximum consecutive consonant run
    max_consec = 0
    cur = 0
    for c in s:
        if c in CONSONANTS:
            cur += 1
            max_consec = max(max_consec, cur)
        else:
            cur = 0
    consecutive_consonants = max_consec

    # Binary: contains at least one digit
    has_numbers = int(digits > 0)

    return {
        'length': length,
        'entropy': entropy,
        'digit_ratio': digit_ratio,
        'vowel_ratio': vowel_ratio,
        'consonant_ratio': consonant_ratio,
        'unique_chars': unique_chars,
        'consecutive_consonants': consecutive_consonants,
        'has_numbers': has_numbers
    }


# ── Apply to dataset ──────────────────────────────────────────────────────────
print('Stripping TLDs ...')
df['sld'] = df['domain'].apply(strip_tld)

print('Extracting features (this may take ~30 seconds for large datasets) ...')
features_list = df['sld'].apply(extract_features)
features_df   = pd.DataFrame(features_list.tolist())

# Attach labels
features_df['label'] = df['label'].values

print(f'\nFeature matrix shape: {features_df.shape}')
print('\nSample feature rows:')
features_df.head()
```

Team: Analytics Shield

Output:

Stripping TLDs ...
Extracting features (this may take ~30 seconds for large datasets) ...

Feature matrix shape: (70000, 9)

Sample feature rows:

	length	entropy	digit_ratio	vowel_ratio	consonant_ratio	unique_chars	consecutive_consonants	has_numbers	label
0	11	3.095795	0.0	0.272727	0.636364	9	3	0	0
1	5	1.921928	0.0	0.600000	0.400000	4	1	0	0
2	8	2.500000	0.0	0.500000	0.500000	6	1	0	0
3	9	2.947703	0.0	0.444444	0.555556	8	2	0	0
4	11	3.095795	0.0	0.272727	0.636364	9	2	0	0

Section 4 – Preprocessing

```
FEATURE_COLS = ['length', 'entropy', 'digit_ratio', 'vowel_ratio',
                'consonant_ratio', 'unique_chars', 'consecutive_consonants', 'has_numbers']

# ── Step a: Remove null rows ──────────────────────────────────────────
pre = features_df[FEATURE_COLS + ['label']].dropna().reset_index(drop=True)
print(f'After removing null rows    : {pre.shape}')

X_raw = pre[FEATURE_COLS].values
y_raw = pre['label'].values
```

Output: After removing null rows   : (70000, 9)

Team: Analytics Shield

```
# ── Step b: VarianceThreshold — remove low-variance features ──────────────
vt = VarianceThreshold(threshold=0.0)   # removes only perfectly constant features
X_vt = vt.fit_transform(X_raw)
retained_features = [FEATURE_COLS[i] for i in range(len(FEATURE_COLS)) if
vt.get_support()[i]]
print(f'After VarianceThreshold    : {X_vt.shape}  |  Retained features: {retained_features}')
```

Output: After VarianceThreshold : (70000, 8) | Retained features: ['length', 'entropy', 'digit_ratio', 'vowel_ratio', 'consonant_ratio', 'unique_chars', 'consecutive_consonants', 'has_numbers']

```
# ── Step c: StandardScaler — normalise to zero mean / unit variance ────────
scaler = StandardScaler()
X_scaled = scaler.fit_transform(X_vt)
print(f'After StandardScaler       : {X_scaled.shape}')
```

Output: After StandardScaler : (70000, 8)

```
# ── Step d: LabelEncoder — confirm integer labels ──────────────────────────
le = LabelEncoder()
y = le.fit_transform(y_raw)
print(f'Label classes              : {le.classes_}')
print(f'Final feature matrix shape  : {X_scaled.shape}')
print(f'Final label vector shape    : {y.shape}')

# X_scaled and y are the canonical preprocessed dataset used throughout
X = X_scaled
```

Output:

Label classes : [0 1]

Final feature matrix shape : (70000, 8)
Final label vector shape : (70000,)

Section 5 - PCA: Dimensionality Reduction & Variance Analysis

```
# ── Full PCA to compute explained variance ─────────────────────────────────
n_features = X.shape[1]
pca_full = PCA(n_components=n_features, random_state=SEED)
```

```
pca_full.fit(X)

cumvar = np.cumsum(pca_full.explained_variance_ratio_) * 100  # in percent
components = np.arange(1, n_features + 1)

# Find component counts that first reach 87%, 95%, 99%
thresholds = {'87%': 87, '95%': 95, '99%': 99}
threshold_components = {}
for label_t, thr in thresholds.items():
   idx = np.argmax(cumvar >= thr)   # first index where threshold is reached
   threshold_components[label_t] = (idx + 1, cumvar[idx])

print('Cumulative explained variance:')
for i, v in enumerate(cumvar):
   print(f'  PC{i+1}: {v:.2f}%')

print('\nComponents needed per threshold:')
for t, (n_comp, var) in threshold_components.items():
   print(f'  {t} variance → {n_comp} components ({var:.2f}%)')
```

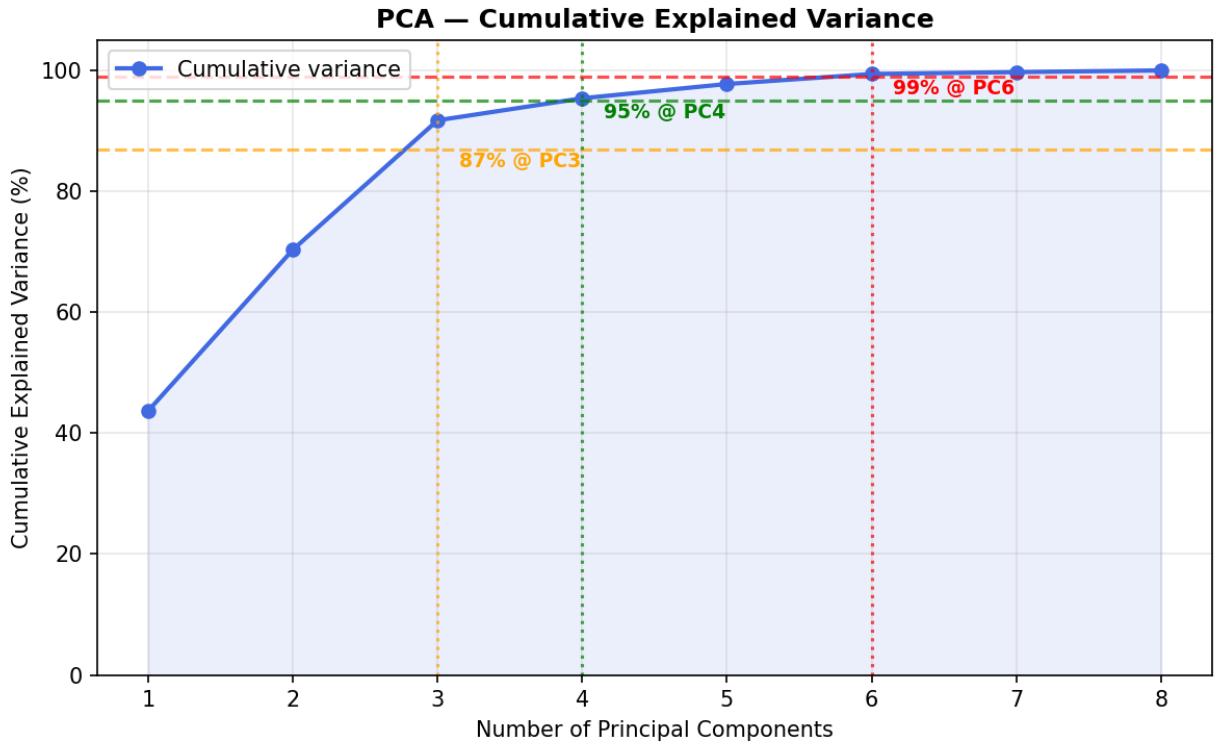
Output:

Cumulative explained variance:

```
  PC1: 43.70%
  PC2: 70.36%
  PC3: 91.75%
  PC4: 95.39%
  PC5: 97.75%
  PC6: 99.41%
  PC7: 99.72%
  PC8: 100.00%

Components needed per threshold:
  87% variance → 3 components (91.75%)
  95% variance → 4 components (95.39%)
  99% variance → 6 components (99.41%)
```

```
# — Figure 3: Cumulative explained variance curve ————————————————
fig, ax = plt.subplots(figsize=(8, 5))
ax.plot(components, cumvar, marker='o', color='royalblue', linewidth=2, label='Cumulative
variance')
ax.fill_between(components, cumvar, alpha=0.1, color='royalblue')
```

```
# Mark threshold lines
colors_t = {'87%': 'orange', '95%': 'green', '99%': 'red'}
for t, (n_comp, var) in threshold_components.items():
    ax.axhline(y=thresholds[t], color=colors_t[t], linestyle='--', alpha=0.7)
    ax.axvline(x=n_comp, color=colors_t[t], linestyle=':', alpha=0.7)
    ax.annotate(f'{t} @ PC{n_comp}',
                xy=(n_comp, thresholds[t]),
                xytext=(n_comp + 0.15, thresholds[t] - 3),
                fontsize=9, color=colors_t[t], fontweight='bold')

ax.set_xlabel('Number of Principal Components')
ax.set_ylabel('Cumulative Explained Variance (%)')
ax.set_title('PCA - Cumulative Explained Variance', fontweight='bold')
ax.set_xticks(components)
ax.set_ylim([0, 105])
ax.grid(True, alpha=0.3)
ax.legend()
plt.tight_layout()
plt.savefig('results/03_pca_variance.png', dpi=150)
plt.show()
print('Saved: results/03_pca_variance.png')
```




```
# ── Create three PCA-projected feature sets ──────────────────────────
```

```
# n_components cannot exceed number of features; cap both at n_features
n_pca_10 = min(10, n_features)
n_pca_25 = min(25, n_features)

pca_10 = PCA(n_components=n_pca_10, random_state=SEED)
X_pca_10 = pca_10.fit_transform(X)

pca_25 = PCA(n_components=n_pca_25, random_state=SEED)
X_pca_25 = pca_25.fit_transform(X)

X_full = X   # no PCA reduction

print(f'X_pca_10 shape : {X_pca_10.shape}')
print(f'X_pca_25 shape : {X_pca_25.shape}')
print(f'X_full   shape : {X_full.shape}')
```

Output:

```
X_pca_10 shape : (70000, 8)

X_pca_25 shape : (70000, 8)
X_full   shape : (70000, 8)
```

Section 6 - Model Training & Evaluation

```
# ── Model factory ──────────────────────────────────────────────────────
def get_models():
    """Return a dict of model name → instantiated sklearn estimator."""
    return {
        'Decision Tree': DecisionTreeClassifier(max_depth=10, random_state=SEED),
        'Random Forest': RandomForestClassifier(n_estimators=100, max_depth=10,
                                                random_state=SEED, n_jobs=-1),
        'SVM':           SVC(kernel='rbf', C=1.0, max_iter=100, random_state=SEED),
        'ANN/MLP':       MLPClassifier(hidden_layer_sizes=(64, 32),
                                       max_iter=300, random_state=SEED)
    }

# ── Split configurations: (test_size, label) ──────────────────────────
SPLITS = [
    (0.20, '80:20'),
    (0.33, '67:33'),
    (0.50, '50:50'),
    (0.05, '95:5'),
```

```
]

# — Training loop ————————————————————————————————————————
results = []   # collect per-model per-split metrics

# Also store fitted models + predictions for 80:20 split (used in Section 7)
models_8020   = {}
preds_8020    = {}
y_test_8020   = None

for test_size, split_label in SPLITS:
    # Create the split once per configuration (same split for all models)
    X_tr, X_te, y_tr, y_te = train_test_split(
        X_full, y, test_size=test_size, random_state=SEED, stratify=y
    )

    if split_label == '80:20':
        y_test_8020 = y_te   # save test labels for confusion matrices

    print(f'\n--- Split {split_label}  (train={len(y_tr):,}, test={len(y_te):,}) ---')

    for model_name, model in get_models().items():
        # Train
        t0 = time.time()
        model.fit(X_tr, y_tr)
        train_time = time.time() - t0

        # Predict
        y_pred = model.predict(X_te)

        # Metrics
        acc  = accuracy_score(y_te, y_pred)
        prec = precision_score(y_te, y_pred, zero_division=0)
        rec  = recall_score(y_te, y_pred, zero_division=0)
        f1   = f1_score(y_te, y_pred, zero_division=0)

        results.append({
            'Model':       model_name,
            'Split':       split_label,
            'Accuracy':    round(acc,  4),
            'Precision':   round(prec, 4),
            'Recall':      round(rec,  4),
            'F1':          round(f1,   4),
            'Training_Time': round(train_time, 3)
```

```
        })

        print(f'  {model_name:<18} Acc={acc:.4f}  P={prec:.4f}  R={rec:.4f}  F1={f1:.4f}
t={train_time:.2f}s')

        # Save models/preds for the 80:20 split
        if split_label == '80:20':
            models_8020[model_name] = model
            preds_8020[model_name]  = y_pred

results_df = pd.DataFrame(results)
print('\nTraining complete.')
```

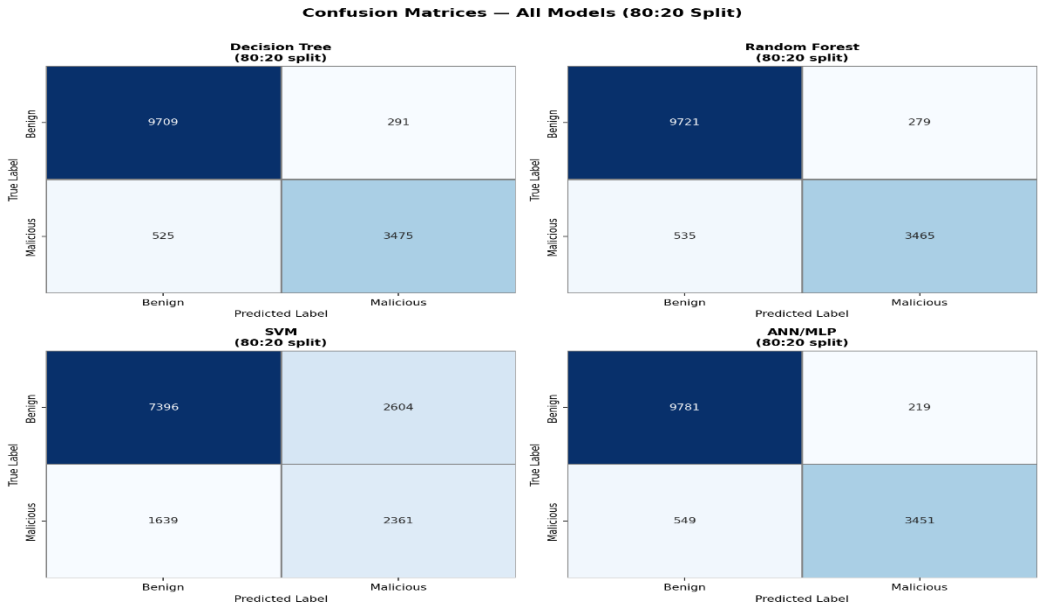
Output:

```
--- Split 80:20  (train=56,000, test=14,000) ---
  Decision Tree      Acc=0.9417  P=0.9227  R=0.8688  F1=0.8949  t=0.21s
  Random Forest      Acc=0.9419  P=0.9255  R=0.8662  F1=0.8949  t=1.15s
  SVM                Acc=0.6969  P=0.4755  R=0.5903  F1=0.5267  t=1.93s
  ANN/MLP            Acc=0.9451  P=0.9403  R=0.8628  F1=0.8999  t=87.91s

--- Split 67:33  (train=46,900, test=23,100) ---
  Decision Tree      Acc=0.9397  P=0.9189  R=0.8653  F1=0.8913  t=0.20s
  Random Forest      Acc=0.9430  P=0.9286  R=0.8673  F1=0.8969  t=0.98s
  SVM                Acc=0.7349  P=0.5277  R=0.6876  F1=0.5971  t=1.64s
  ANN/MLP            Acc=0.9440  P=0.9507  R=0.8479  F1=0.8964  t=106.32s

--- Split 50:50  (train=35,000, test=35,000) ---
  Decision Tree      Acc=0.9375  P=0.9113  R=0.8656  F1=0.8878  t=0.12s
  Random Forest      Acc=0.9399  P=0.9258  R=0.8584  F1=0.8908  t=0.75s
  SVM                Acc=0.7571  P=0.5640  R=0.6606  F1=0.6085  t=0.91s
  ANN/MLP            Acc=0.9421  P=0.9300  R=0.8624  F1=0.8949  t=64.73s

--- Split 95:5  (train=66,500, test=3,500) ---
  Decision Tree      Acc=0.9343  P=0.9456  R=0.8170  F1=0.8766  t=0.30s
  Random Forest      Acc=0.9391  P=0.9377  R=0.8430  F1=0.8878  t=1.27s
  SVM                Acc=0.7663  P=0.6123  R=0.4960  F1=0.5481  t=2.49s
  ANN/MLP            Acc=0.9400  P=0.9238  R=0.8610  F1=0.8913  t=121.92s

Training complete.
```

Team: Analytics Shield

Section 7 - Confusion Matrices

```
# ── Figure 4: 2×2 grid of confusion matrix heatmaps (80:20 split) ──────────
model_names = list(preds_8020.keys())
fig, axes = plt.subplots(2, 2, figsize=(12, 10))
axes_flat = axes.flatten()

class_names = ['Benign', 'Malicious']

for idx, model_name in enumerate(model_names):
    cm = confusion_matrix(y_test_8020, preds_8020[model_name])
    ax = axes_flat[idx]

    sns.heatmap(cm, annot=True, fmt='d', cmap='Blues', ax=ax,
                xticklabels=class_names, yticklabels=class_names,
                linewidths=0.5, linecolor='gray', cbar=False)

    # Annotate TP / TN in green, FP / FN in red
    ax.set_title(f'{model_name}\n(80:20 split)', fontweight='bold', fontsize=11)
    ax.set_xlabel('Predicted Label')
    ax.set_ylabel('True Label')

plt.suptitle('Confusion Matrices - All Models (80:20 Split)',
             fontsize=14, fontweight='bold', y=1.01)
plt.tight_layout()
plt.savefig('results/04_confusion_matrices.png', dpi=150, bbox_inches='tight')
plt.show()
print('Saved: results/04_confusion_matrices.png')
```



Section 8 - PCA Feature-Set Comparison

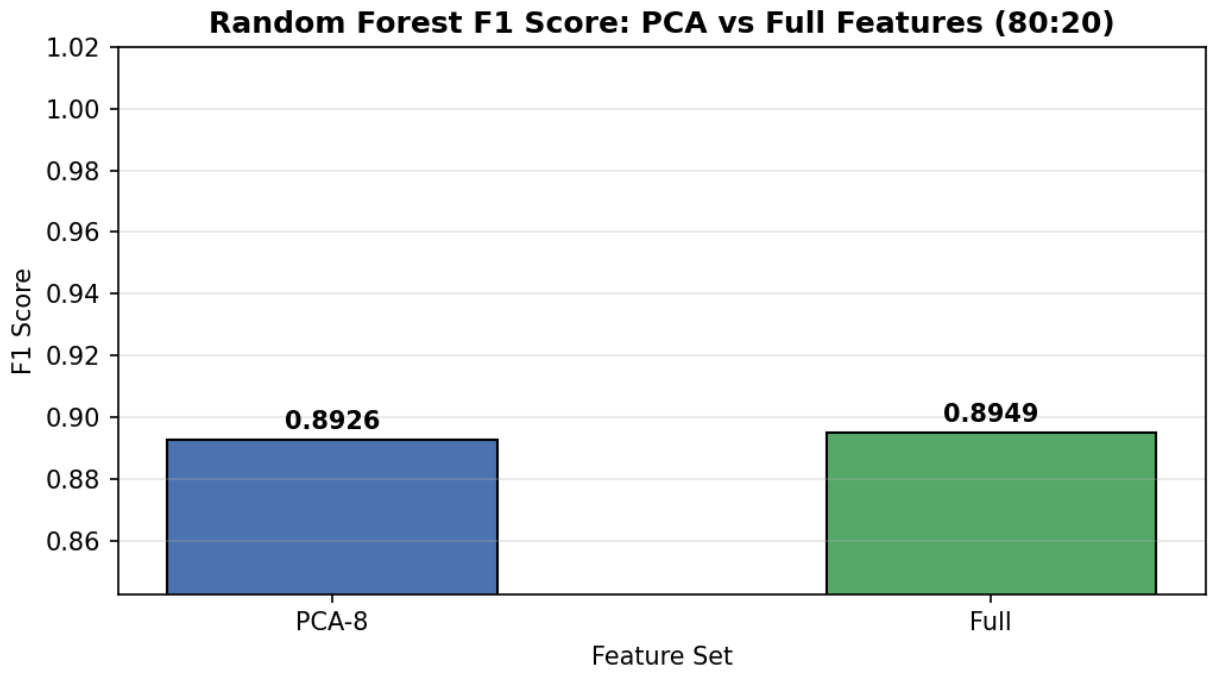
```
# ── Train Random Forest on each PCA variant (80:20) ──────────────────────
pca_comparison = {}
feature_sets = {
  f'PCA-{n_pca_10}': X_pca_10,
  f'PCA-{n_pca_25}': X_pca_25,
  'Full':          X_full
}

for fs_name, X_fs in feature_sets.items():
  X_tr, X_te, y_tr, y_te = train_test_split(
    X_fs, y, test_size=0.20, random_state=SEED, stratify=y
  )
  rf = RandomForestClassifier(n_estimators=100, max_depth=10,
                              random_state=SEED, n_jobs=-1)
  rf.fit(X_tr, y_tr)
  y_pred = rf.predict(X_te)
  f1 = f1_score(y_te, y_pred, zero_division=0)
  pca_comparison[fs_name] = round(f1, 4)
  print(f'  Random Forest | {fs_name:<10} | F1 = {f1:.4f}')

print()
```

Output:

```
Random Forest | PCA-8     | F1 = 0.8926
  Random Forest | Full      | F1 = 0.8949
```

```
# ── Figure 5: PCA comparison bar chart ──────────────────────────────────
fig, ax = plt.subplots(figsize=(7, 4))
names  = list(pca_comparison.keys())
scores = list(pca_comparison.values())
palette = ['#4C72B0', '#55A868', '#C44E52']

bars = ax.bar(names, scores, color=palette, edgecolor='black', width=0.5)
for bar, val in zip(bars, scores):
  ax.text(bar.get_x() + bar.get_width() / 2,
      bar.get_height() + 0.002,
      f'{val:.4f}', ha='center', va='bottom', fontsize=10, fontweight='bold')

ax.set_ylim([max(0, min(scores) - 0.05), 1.02])
ax.set_title('Random Forest F1 Score: PCA vs Full Features (80:20)',
```

```
        fontweight='bold')
ax.set_xlabel('Feature Set')
ax.set_ylabel('F1 Score')
ax.grid(axis='y', alpha=0.3)
plt.tight_layout()
plt.savefig('results/05_pca_comparison.png', dpi=150)
plt.show()
print('Saved: results/05_pca_comparison.png')
```



Section 9 - Feature Importance

```
# — Extract feature importances from the 80:20 Random Forest model ————
rf_model = models_8020['Random Forest']
importances = rf_model.feature_importances_

# Map to retained feature names (after VarianceThreshold)
fi_df = pd.DataFrame({
   'feature':    retained_features,
   'importance': importances
}).sort_values('importance', ascending=True)   # ascending for horizontal bar

print('Feature importances (sorted):')
print(fi_df.sort_values('importance', ascending=False).to_string(index=False))
```

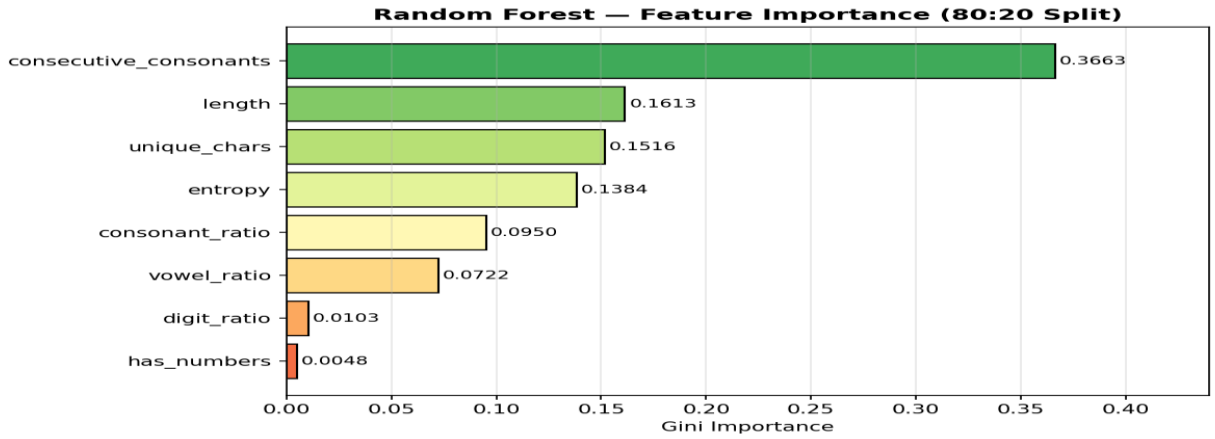
Team: Analytics Shield

Output:

Feature importances (sorted):

feature	importance
consecutive_consonants	0.366297
length	0.161347
unique_chars	0.151570
entropy	0.138409
consonant_ratio	0.095018
vowel_ratio	0.072208
digit_ratio	0.010302
has_numbers	0.004849

```
# ── Figure 6: Horizontal feature importance bar chart ──────────────────
fig, ax = plt.subplots(figsize=(8, 5))
colors_fi = plt.cm.RdYlGn(np.linspace(0.2, 0.85, len(fi_df)))
ax.barh(fi_df['feature'], fi_df['importance'], color=colors_fi, edgecolor='black')

for i, (val, feat) in enumerate(zip(fi_df['importance'], fi_df['feature'])):
    ax.text(val + 0.002, i, f'{val:.4f}', va='center', fontsize=9)

ax.set_xlabel('Gini Importance')
ax.set_title('Random Forest - Feature Importance (80:20 Split)',
             fontweight='bold')
ax.set_xlim([0, max(fi_df['importance']) * 1.2])
ax.grid(axis='x', alpha=0.3)
plt.tight_layout()
plt.savefig('results/06_feature_importance.png', dpi=150)
plt.show()
print('Saved: results/06_feature_importance.png')
```



Section 10 - Final Summary Table

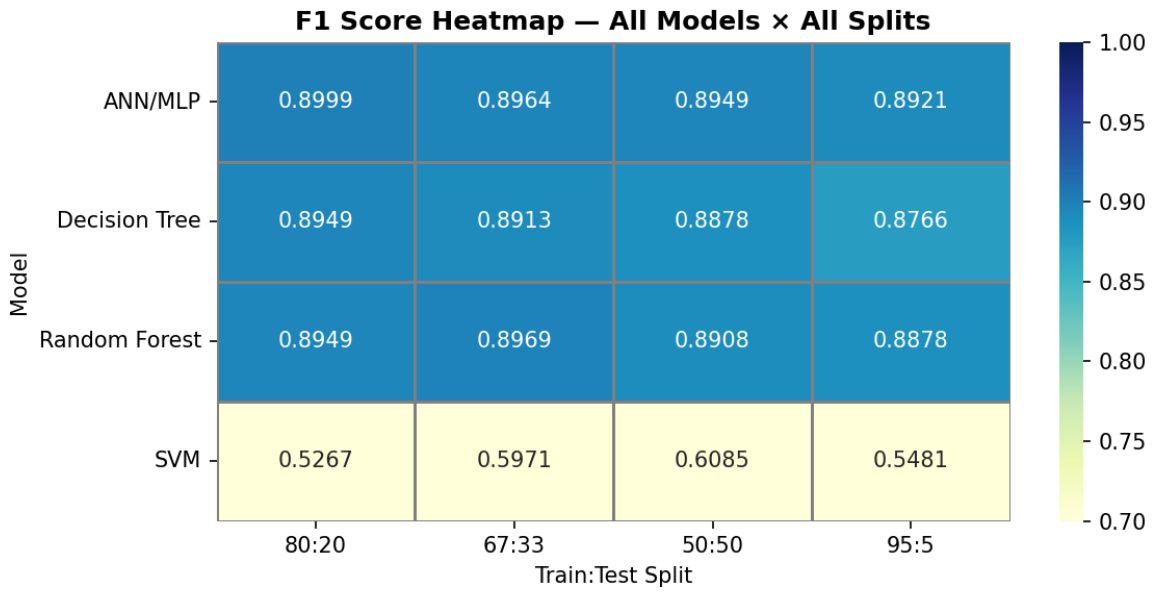
```
# ── Build and display final summary table ──────────────────────────────
summary = results_df[['Model', 'Split', 'Accuracy', 'Precision', 'Recall',
                      'F1', 'Training_Time']].sort_values('F1', ascending=False)

print('='*80)
print('FINAL SUMMARY TABLE — All Models × All Splits (sorted by F1 descending)')
print('='*80)
print(summary.to_string(index=False))
```

Output:

```
================================================================================
FINAL SUMMARY TABLE — All Models × All Splits (sorted by F1 descending)
================================================================================
      Model Split  Accuracy  Precision  Recall    F1  Training_Time
    ANN/MLP 80:20    0.9451     0.9403  0.8628 0.8999       87.912
Random Forest 67:33    0.9430     0.9286  0.8673 0.8969        0.982
    ANN/MLP 67:33    0.9440     0.9507  0.8479 0.8964      106.323
Decision Tree 80:20    0.9417     0.9227  0.8688 0.8949        0.209
Random Forest 80:20    0.9419     0.9255  0.8662 0.8949        1.151
    ANN/MLP 50:50    0.9421     0.9300  0.8624 0.8949       64.727
Decision Tree 67:33    0.9397     0.9189  0.8653 0.8913        0.199
    ANN/MLP  95:5    0.9400     0.9238  0.8610 0.8913      121.923
Random Forest 50:50    0.9399     0.9258  0.8584 0.8908        0.752
Decision Tree 50:50    0.9375     0.9113  0.8656 0.8878        0.125
Random Forest  95:5    0.9391     0.9377  0.8430 0.8878        1.269
Decision Tree  95:5    0.9343     0.9456  0.8170 0.8766        0.299
       SVM 50:50    0.7571     0.5640  0.6606 0.6085        0.910
       SVM 67:33    0.7349     0.5277  0.6876 0.5971        1.644
       SVM  95:5    0.7663     0.6123  0.4960 0.5481        2.494
       SVM 80:20    0.6969     0.4755  0.5903 0.5267        1.935
```

```
# ── Figure 7: Summary heatmap — F1 scores ──────────────────────────────
pivot_f1 = results_df.pivot(index='Model', columns='Split', values='F1')
# Reorder columns logically
col_order = [c for c in ['80:20', '67:33', '50:50', '95:5'] if c in pivot_f1.columns]
pivot_f1 = pivot_f1[col_order]

fig, ax = plt.subplots(figsize=(8, 4))
sns.heatmap(pivot_f1, annot=True, fmt='.4f', cmap='YlGnBu', ax=ax,
        linewidths=0.5, linecolor='gray', vmin=0.7, vmax=1.0)
ax.set_title('F1 Score Heatmap - All Models × All Splits', fontweight='bold')
ax.set_xlabel('Train:Test Split')
```

```
ax.set_ylabel('Model')
plt.tight_layout()
plt.savefig('results/07_f1_heatmap.png', dpi=150)
plt.show()
print('Saved: results/07_f1_heatmap.png')
```




```
# — PROJECT COMPLETE — identify best model ————————————————————————
best_row = results_df.loc[results_df['F1'].idxmax()]
best_model_name = best_row['Model']
best_f1       = best_row['F1']
best_split    = best_row['Split']

print('\n' + '='*60)
print(f'  Best Model    : {best_model_name}')
print(f'  Best Split    : {best_split}')
print(f'  Best F1 Score : {best_f1:.4f}')
print('  PROJECT COMPLETE')
print('='*60)
print(f'\nAll plots saved in: ./results/')
saved_plots = sorted(os.listdir('results'))
for p in saved_plots:
    print(f'  {p}')
```

Output:

```
============================================================
  Best Model    : ANN/MLP
```

Team: Analytics Shield

Best Split : 80:20
Best F1 Score : 0.8999
PROJECT COMPLETE
==========